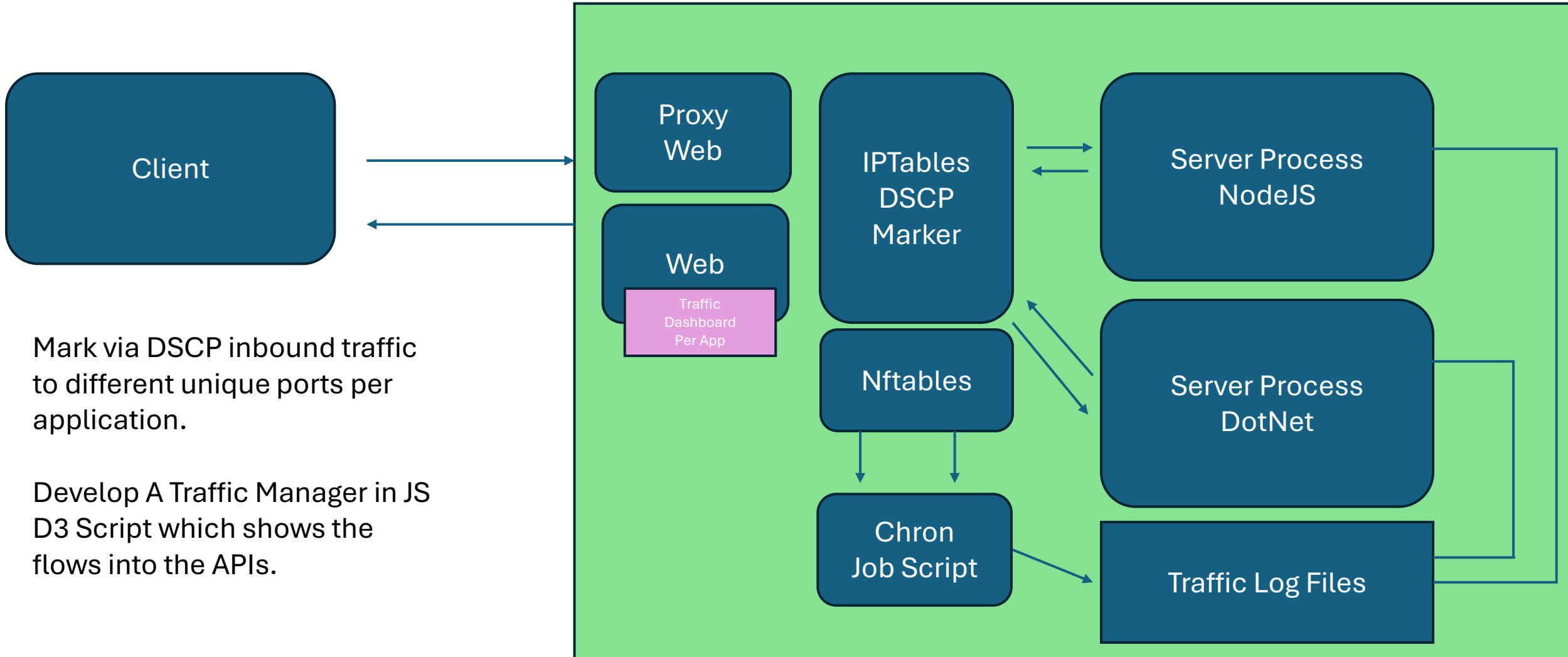


REST Traffic Flow Project

Lab Design For CSCE799

Architectural Design Requirement #8

Flow Design Client Server REST API Traffic Collector (Linux)



IPTables(Linux FW) has the ability to mark traffic destined for a service in Linux...(DSCP inbound)

okay so I want to mark traffic destined to a DOTNET REST API, and a NODEJS instance with a specific DSCP value

Yes. For what you're describing, you generally want to set the DSCP value on ingress based on the destination port of the service.

Assuming:

- .NET REST API → TCP 5000
- NodeJS app → TCP 3000

You can use the Linux mangle table to rewrite the DSCP field as packets arrive.

Using iptables

Mark traffic destined to the .NET API as AF31 (26):

```
1 iptables -t mangle -A PREROUTING \
2 -p tcp --dport 5000 \
3 -j DSCP --set-dscp-class AF31
```

Mark traffic destined to NodeJS as AF21 (18):

```
1 iptables -t mangle -A PREROUTING \
2 -p tcp --dport 3000 \
3 -j DSCP --set-dscp-class AF21
```

Or numeric values:

```
1 iptables -t mangle -A PREROUTING -p tcp --dport 5000 -j DSCP --set-dscp 26
2 iptables -t mangle -A PREROUTING -p tcp --dport 3000 -j DSCP --set-dscp 18
```

If You Want Symmetric Marking

Mark ingress and egress:

```
1 # Incoming to .NET
2 iptables -t mangle -A PREROUTING \
3 -p tcp --dport 5000 \
4 -j DSCP --set-dscp-class AF31
5
6 # Outgoing from .NET
7 iptables -t mangle -A POSTROUTING \
8 -p tcp --sport 5000 \
9 -j DSCP --set-dscp-class AF31
```

Better Approach: Use Connection Tracking

Rather than matching every outbound packet by source port, you can mark the connection once on ingress and restore the mark on egress.

```
1 # Save connection mark
2 iptables -t mangle -A PREROUTING \
3 -p tcp --dport 5000 \
4 -j CONNMARK --set-mark 31
5
6 # Restore for response packets
7 iptables -t mangle -A POSTROUTING \
8 -j CONNMARK --restore-mark
```

Then translate the connection mark into the desired DSCP value.

This is preferred when:

- Multiple services share ports.
- Reverse proxies are involved.
- Load balancers are involved.
- Service ports change.

Inbound....

Symmetric

AI-generated content may be incorrect.

Copilot Recommendation

then nftables counters become very attractive:

```
1 table inet qos {
2   chain postrouting {
3     type filter hook postrouting priority mangle;
4
5     tcp sport 5000 ip dscp set af31 counter
6     tcp sport 3000 ip dscp set af21 counter
7   }
8 }
```

Now:

```
1 nft list ruleset
```

might show:

```
1 AF31:
2   packets 1500000
3   bytes   24G
4
5 AF21:
6   packets 280000
7   bytes   4G
```

which directly answers:

- Total .NET API traffic = 24 GB
- Total NodeJS traffic = 4 GB

without deploying a full NetFlow collector.

My Recommendation

For a single Linux server running .NET and NodeJS:

1. Set DSCP using nftables in postrouting.
2. Add nftables counters on those same rules.
3. Export metrics to Prometheus/Grafana if you want dashboards.

+ Message Copilot

My Recommendation

For a single Linux server running .NET and NodeJS:

1. Set DSCP using nftables in postrouting.
2. Add nftables counters on those same rules.
3. Export metrics to Prometheus/Grafana if you want dashboards.

That's lightweight, built into Linux, and gives accurate byte and packet counts per DSCP-marked application flow.

👍 🤔 ...

okay I want to write the traffic out to a text file on a per day basis in bytes, and then use nodejs, and C# scripts to import the log files

That's actually a pretty clean architecture.

Instead of capturing every flow with NetFlow, I'd use nftables counters and export a daily snapshot to a text file. Then both your NodeJS and C# applications can import the same flat-file format.

Option 1: Daily DSCP Counter Log

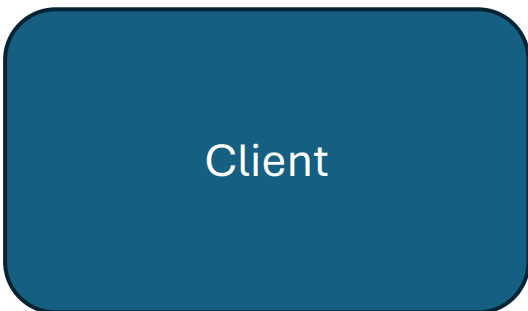
Create DSCP counters:

```
1 table inet qos {
2   chain postrouting {
3     type filter hook postrouting priority mangle;
4
5     tcp sport 5000 ip dscp set af31 counter name dotnet_api
6     tcp sport 3000 ip dscp set af21 counter name node_api
7   }
8 }
9 ..
```

A script can dump the counters once per day:

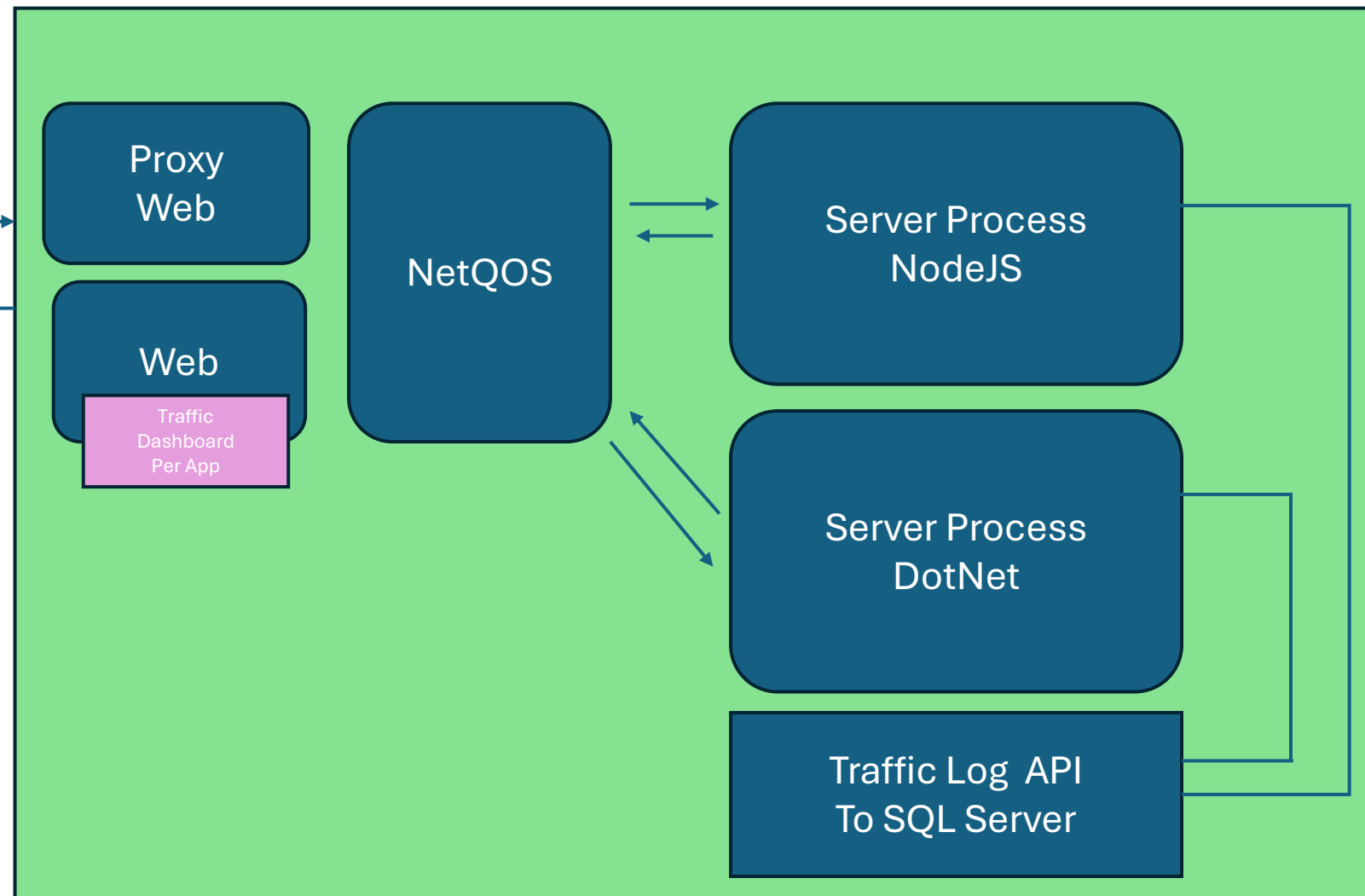
+ Message Copilot

Flow Design Client Server REST API Traffic Collector (Windows)



Windows has programmatic access to NetQOS templates and counters.

Thus NodeJS and DotNet can access the historical data directly without logs. OR we could keep the architecture the same, and dump identical files to disk before import as a unified process.



Summary of Windows Process

There are three practical approaches:

Option 1: Windows QoS Policies (Recommended)

Windows can mark traffic generated by a specific executable with a DSCP value.

For example:

.NET API

```
1 New-NetQosPolicy -
2   -Name "DotNetAPI" -
3   -AppPathNameMatchCondition "C:\Apps\Api\MyApi.exe" -
4   -DSCPAction 26
```

NodeJS

```
1 New-NetQosPolicy -
2   -Name "NodeJS" -
3   -AppPathNameMatchCondition "C:\Program Files\nodejs\node.exe" -
4   -DSCPAction 18
```

You can also target ports:

```
1 New-NetQosPolicy -
2   -Name "DotNet5000" -
3   -IPPortMatchCondition 5000 -
4   -IPProtocol TCP -
5   -DSCPAction 26
```

Then verify:

```
1 Get-NetQosPolicy
```

This gives you DSCP tagging similar to your Linux example.

Option 2: Windows Performance Counters

Windows exposes per-process network statistics.

Examples:

```
1 Get-Counter "\\Process(MyApi)\IO Read Bytes/sec"
```

or

```
1 Get-Counter "\\Network Interface(*)\Bytes Total/sec"
```

A scheduled task could write daily totals:

```
1 2026-09-25,dotnet_api,1843922123
2 2026-09-25,node_api,628312332
```

Write to clipboard or share to OneDrive and OneNote

+ Message Copilot

Services Required

- Linux

- Iptables
- Nftables
- DotNet SystemD
- NodeJS SystemD
- Wireshark in Promiscuous Mode for Validation
- Custom Encryption Algorithms.

- Windows

- NetQOS configuration
- Chron Process As A Services
- DotNet Service Provisioning
- NodeJS Service Provisioning
- Wireshark
- Promiscuous Mode Network Drivers.

Cisco SPAN Port Type to install a probe into a VLAN.

Cisco calls this Switched Port Analyzer (SPAN).

Example:



The image shows a code editor window with a toolbar at the top containing icons for search, copy, paste, and a dropdown menu. A button labeled '</> Plain Text' is also present. A notification bar at the top of the editor area states: 'cisco isn't fully supported. Syntax highlighting is based on Plain Text.' The main area of the editor contains the following configuration commands:

```
1 monitor session 1 source vlan 100
2 monitor session 1 destination interface Gi1/0/24
3`
```

This copies all traffic from VLAN 100 to port Gi1/0/24 where you can connect:

- Wireshark
- IDS/IPS
- Traffic collector
- Linux sensor

This is usually what people mean when they want to "see all VLAN traffic."

Admin Rights

- I need root access to both servers for at least one login for at least two weeks from delivery for software installation.
- I don't mind a change of control after that point in time.